

Performance Testing

Date	15/07/2026
Team ID	SWTID-2026-3095
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

This measures actual application and model latency (request parsing, rule evaluation, XGBoost inference, SHAP explanation, template rendering) without network overhead. Formal external load testing (e.g. with Locust or JMeter against a hosted deployment) is listed as future work in Folder 8: Scalability & Future Plan.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Flask test client (in-process) + Python's time / concurrent.futures modules
Type of Testing	Functional latency testing + a lightweight concurrent-throughput test
Target Module / API	POST /predict (both the rule-gate decline path and the full ML+SHAP path), GET / (home dashboard)
Test Environment	Local (in-process; the app is not yet hosted, so this is component-level rather than network-level testing)
Test Date	07 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Use	Duration (sec)	Expected Outcome
1	Repeated GET / (home dashboard)	1 (sequential)	n/a — 200 requests timed	Sub-millisecond response; static metrics read from memory
2	Repeated POST /predict — rule-gate decline (income=0)	1 (sequential)	n/a — 200 requests timed	Near-instant; short-circuits before any ML call
3	Repeated POST /predict — full ML + SHAP path	1 (sequential)	n/a — 200 requests timed	Fast enough to feel instant to a human user (well under 100ms)
4	Concurrent POST /predict — full ML + SHAP path	8 (thread pool)	~4.6s for 300 requests	System remains responsive with no errors under light concurrency

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status
1	Response Time (Avg) — Home page	< 2 seconds	1.86 ms (0.27 ms p50)	Pass
2	Response Time (Avg) — Rule-gate decline	< 2 seconds	0.45 ms (0.43 ms p50)	Pass
3	Response Time (Avg) — Full ML + SHAP scoring	< 2 seconds	15.27 ms (15.03 ms p50)	Pass
4	Response Time (Max) — Full ML + SHAP scoring	< 5 seconds	29.32 ms	Pass
5	Throughput (Req/sec)	> 20 req/sec	~65.7 req/sec (8 threads, in-process, single Python process — GIL-bound)	Pass
6	Error Rate	< 1%	0% (0 failed requests across all scenarios)	Pass